

Sesión 1: Eficiencia

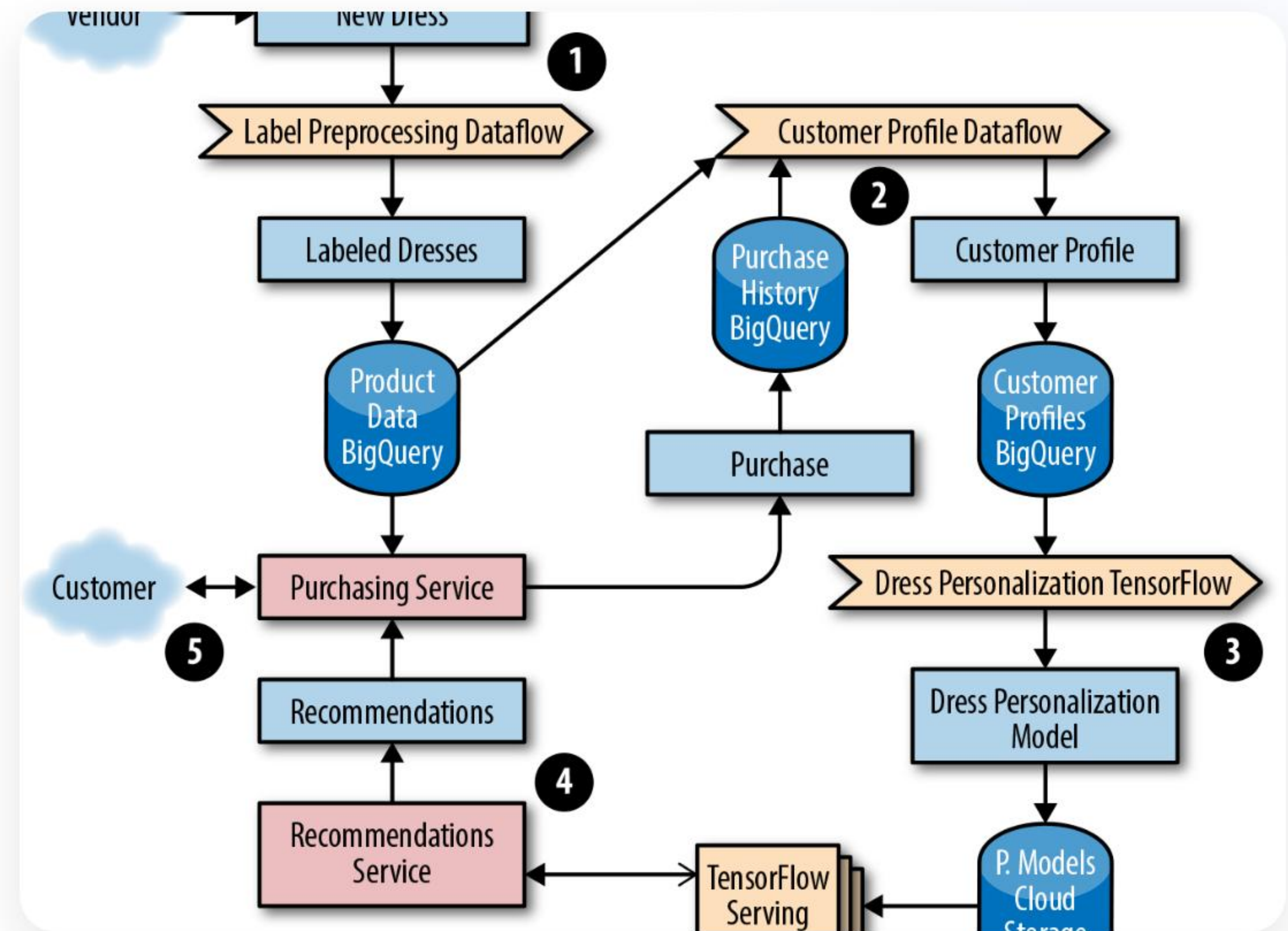
Análisis de Algoritmos y Notación Asintótica en C#

IPC 2 | Facultad de Ingeniería USAC

Laboratorio - Primer Semestre 2026

Agenda de Hoy

- 1 Concepto de Algoritmo y Eficiencia
- 2 Cálculo del tiempo de ejecución $T(n)$
- 3 Notación Big O: Peor y Mejor Caso
- 4 Implementación en C# (.NET)



Fundamentos de Algoritmos

¿Qué hace que un código sea "mejor" que otro?

Definición de Algoritmo

Un conjunto finito de instrucciones precisas y no ambiguas para resolver un problema.



Precisión

Cada paso debe estar definido sin ambigüedades.



Finitud

Debe terminar tras un número finito de pasos.



Entrada/Salida

Recibe datos y produce un resultado esperado.

El concepto de Eficiencia

No basta con que funcione

En IPC 2, el reto es escalar. Un algoritmo puede funcionar para 10 datos, pero ¿qué pasa con 10 millones?

- **Tiempo:** ¿Cuántas operaciones realiza?
- **Espacio:** ¿Cuánta memoria RAM consume?

"La eficiencia es la medida del uso de recursos computacionales por parte de un algoritmo."

Análisis Matemático $T(n)$

Cuantificando el esfuerzo computacional

Midiendo el Tiempo $T(n)$

Representamos el tiempo de ejecución como una función matemática donde n es el tamaño de la entrada.

$$T(n) = \Sigma \text{ operaciones elementales}$$

- Asignaciones, comparaciones y operaciones aritméticas = 1 unidad.
- El tiempo total es la suma de las constantes multiplicadas por el número de veces que se ejecutan.

Ejemplo 1: Tiempo Constante

```
int a = 10; // 1 op int b = 20; // 1 op int suma = a + b; // 2 ops (suma y asignación)  
Console.WriteLine(suma); // 1 op
```

$$T(n) = 1 + 1 + 2 + 1 = 5$$

No depende de n . Siempre tarda lo mismo.

Ejemplo 2: Tiempo Lineal

```
for (int i = 0; i < n; i++) // n + 1 comparaciones { Console.WriteLine(i); // n operaciones }
```

$$T(n) = 1 + (n + 1) + n + n = 3n + 2$$

El tiempo crece proporcionalmente al tamaño de n .

Ejemplo 3: Bucles Anidados

```
for (int i = 0; i < n; i++) { for (int j = 0; j < n; j++) { Console.Write("*"); // Se ejecuta n * n veces } }
```

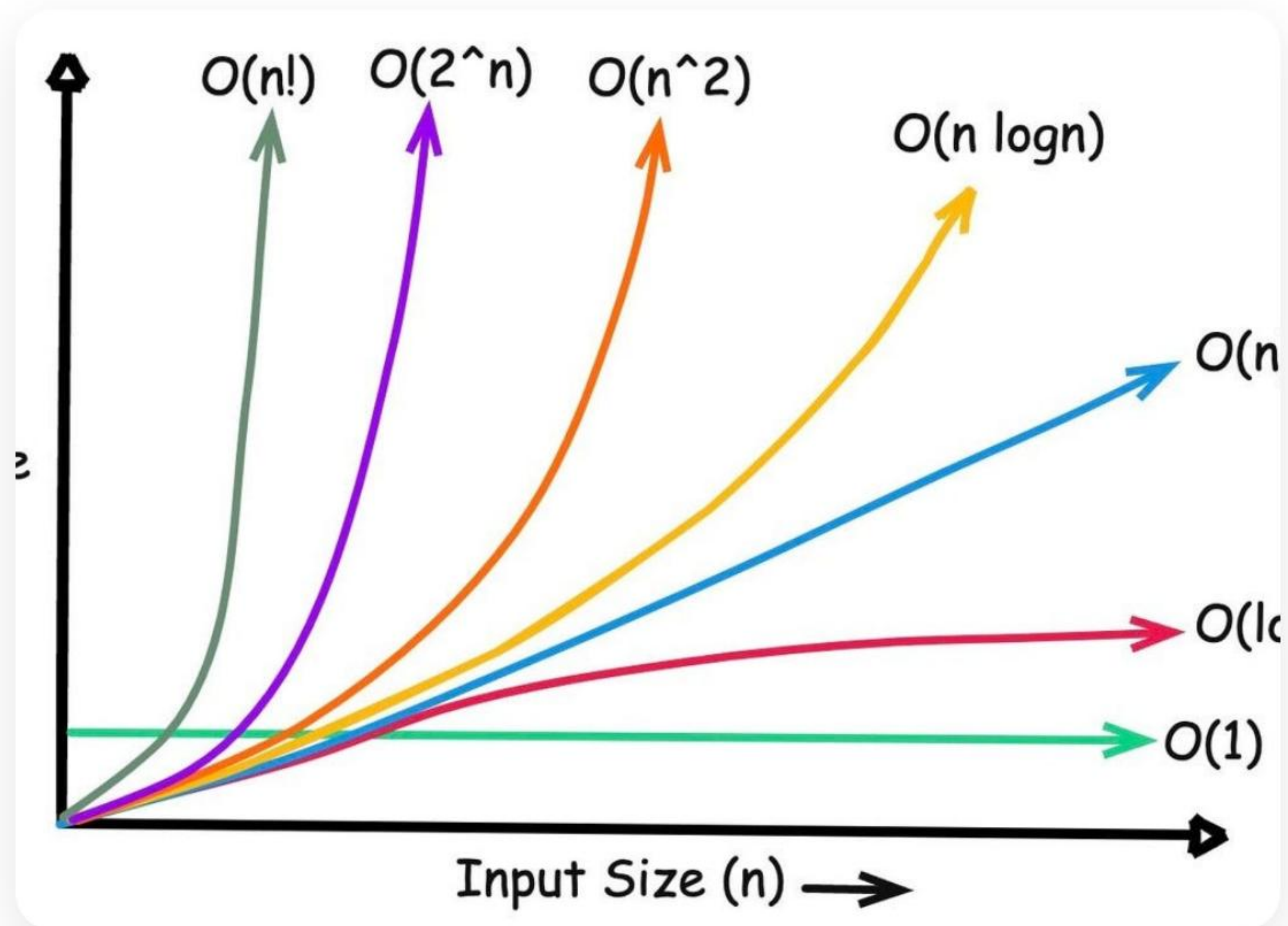
$$T(n) \approx n^2$$

Típico en comparaciones de todos contra todos (ej. Burbuja).

Notación Asintótica (Big O)

Simplificando la complejidad para el mundo real

¿Por qué usar Big O?



Ignoramos las constantes y los términos de menor grado.
Nos interesa el **comportamiento a largo plazo**.

$$3n^2 + 5n + 10 \rightarrow O(n^2)$$

En el análisis asintótico, solo el término que crece más rápido domina la función.

Definición Formal de $O(g(n))$

Decimos que $f(n)$ es $O(g(n))$ si existen constantes c y n_0 tales que:

$$f(n) \leq c \cdot g(n) \text{ para todo } n \geq n_0$$

Representa una **cota superior** (el peor escenario posible).

Peor, Mejor y Promedio



Peor Caso

La cota superior. Notación O . Lo que más nos importa como ingenieros.



Mejor Caso

La cota inferior. Notación Ω (Omega). El escenario ideal.



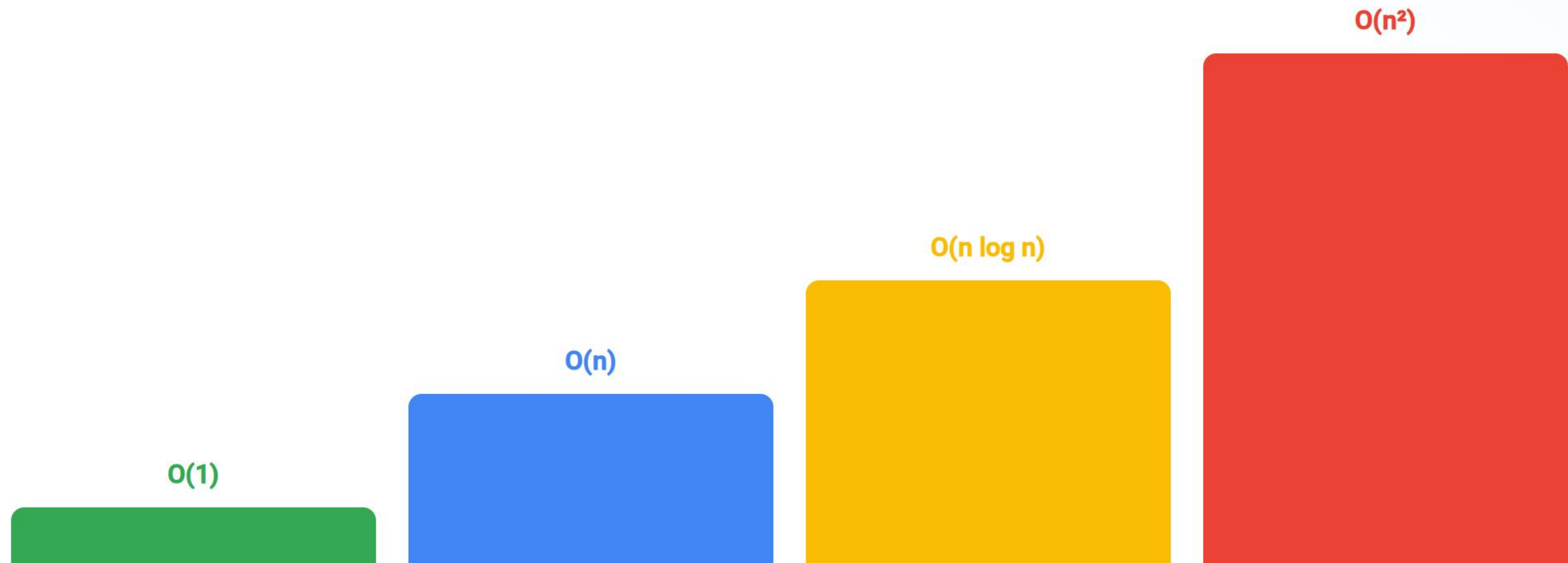
Caso Promedio

Comportamiento esperado. Notación Θ (Theta).

Clases de Complejidad Comunes

Complejidad	Nombre	Ejemplo C#
$O(1)$	Constante	Acceder a un índice de Array
$O(\log n)$	Logarítmico	Búsqueda Binaria
$O(n)$	Lineal	Búsqueda Secuencial
$O(n \log n)$	Lineal-Log	QuickSort / MergeSort
$O(n^2)$	Cuadrático	Bubble Sort
$O(2^n)$	Exponencial	Fibonacci Recursivo

Visualizando el Crecimiento



A medida que n crece, la diferencia entre $O(n)$ y $O(n^2)$ se vuelve abismal.

Reglas para Simplificar Big O

Regla de la Suma

Para bloques secuenciales, tomamos el mayor.

$$O(n) + O(n^2) = \mathbf{O(n^2)}$$

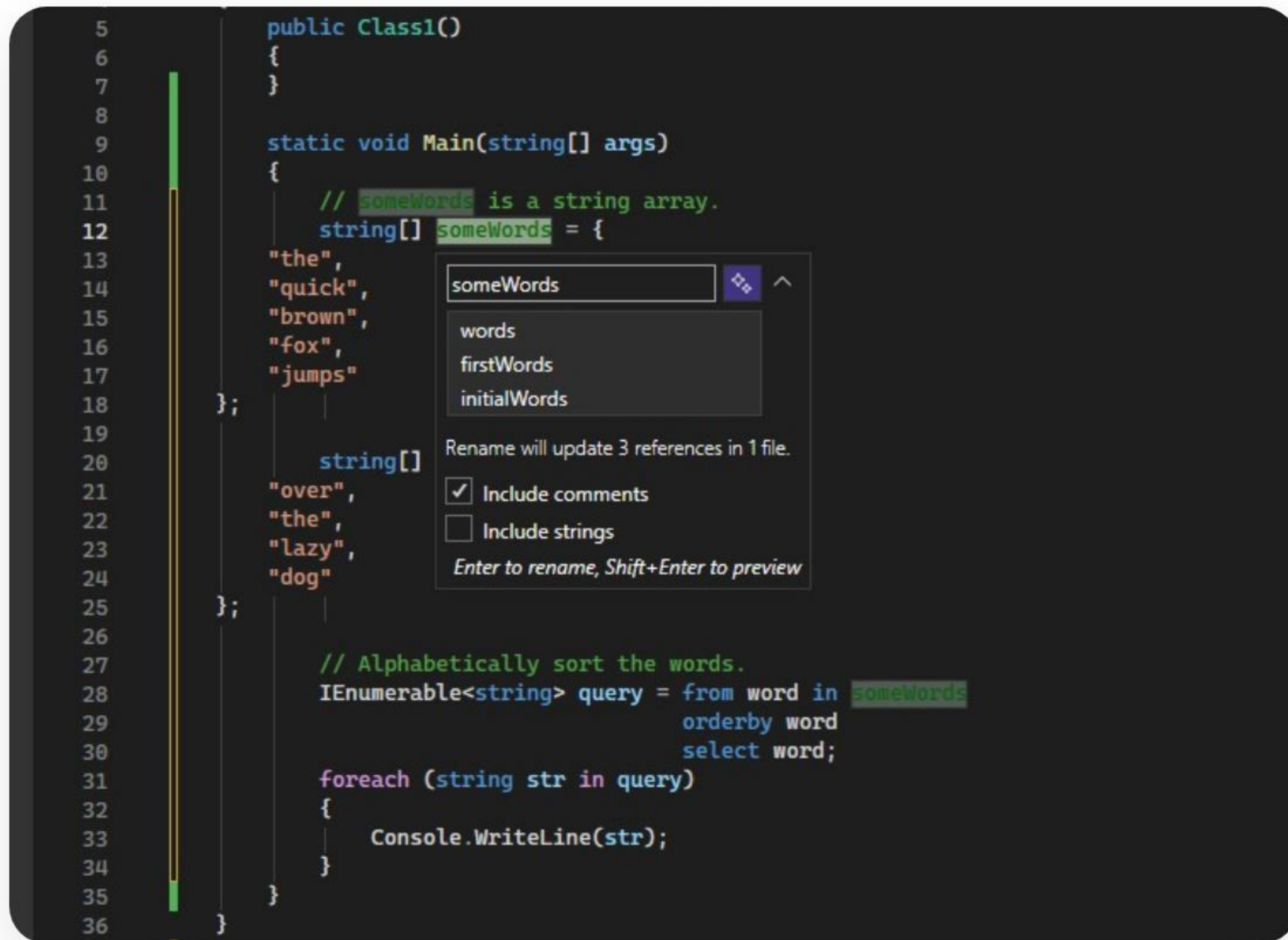
Regla del Producto

Para bloques anidados, multiplicamos.

$$O(n) * O(n) = \mathbf{O(n^2)}$$

Entorno de Trabajo C#

```
5 public Class1()
6 {
7 }
8
9 static void Main(string[] args)
10 {
11     // someWords is a string array.
12     string[] someWords = {
13         "the",
14         "quick",
15         "brown",
16         "fox",
17         "jumps"
18     };
19
20     string[]
21     "over",
22     "the",
23     "lazy",
24     "dog"
25 };
26
27 // Alphabetically sort the words.
28 IEnumerable<string> query = from word in someWords
29                             orderby word
30                             select word;
31
32 foreach (string str in query)
33 {
34     Console.WriteLine(str);
35 }
36 }
```



Análisis Práctico en .NET

En el laboratorio usaremos **Visual Studio 2022** o **VS Code**.

- Utilice Stopwatch para medir tiempos reales.
- Compare resultados experimentales vs análisis teórico Big O.

Ejercicios en C#

Pongamos en práctica el análisis

Ejercicio 1: Búsqueda Lineal

```
bool Buscar(int[] arr, int k) { foreach (int x in arr) { if (x == k) return true; } return false; }
```

Análisis:

- **Mejor caso:** $O(1)$ (está al principio).
- **Peor caso:** $O(n)$ (está al final o no está).

Ejercicio 2: Bucle Logarítmico

```
int i = n; while (i > 1) { i = i / 2; // Se divide a la mitad cada vez Console.WriteLine(i); }
```

Análisis:

Cuando reducimos el espacio de búsqueda o el problema a la mitad en cada paso, la complejidad es **$O(\log n)$** .

Valor de la Unidad: Respeto



"Todos deberían ser respetados como individuos, pero ninguno idealizado."

— Albert Einstein

Fomentamos un ambiente de colaboración y respeto técnico en cada sesión de laboratorio.

Conclusiones

Medición

$T(n)$ nos da el conteo exacto de operaciones.

Abstracción

Big O nos permite comparar algoritmos sin importar el hardware.

C#

El código limpio y eficiente es nuestra meta en este curso.

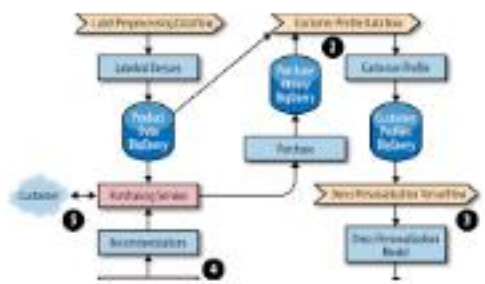
¿Dudas o Preguntas?

¡Gracias por su atención!

 Fernando Vicente: 3021894750101@ingenieria.usac.edu.gt

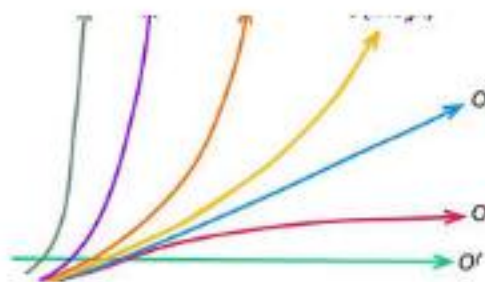
 Plataforma UEDI

Image Sources



https://lh3.googleusercontent.com/vdUfamzd5mee2l60GqUQ320uUWsdLNyoV1EurdYE1DuuMirVPINhi78GHxFZV4kLmXXTzw7BRtmjxAw08DjD_zVWcVXB_I0OsZaR=s1334

Source: [sre.google](#)



https://miro.medium.com/1*ENAP16Z-YXbzEebQlIXFYA.jpeg

Source: [medium.com](#)



<https://learn.microsoft.com/en-us/visualstudio/get-started/media/visualstudio/tutorial-rename-start.png?view=visualstudio>

Source: [learn.microsoft.com](#)